Python OOP and Fundamentals:

Comprehensive Overview

Institute of Technology of Cambodia

CHHEAN RAVY

e20241154

I3_AMS_1A

GROUP 1

Python Basic Data Types

Python provides simple built-in types for single values and powerful collections for organizing data.

Primitive (built-in) types

int for whole numbers, float for decimals, bool for True/False, and str for text.

Collection and reference types

list (ordered, mutable), tuple (ordered, immutable), dict (key-value mappings), set (unique, unordered), and objects (class instances).

Literals in Python

Use literals to write values directly in code: numbers, strings, booleans, None, and collection literals.

```
# Student data using various data types
# and collections

studentNames = ["Alice", "Bob", "Charlie"]
# list of strings

student_ages = (20, 21, 19)
# tuple of ints

student_scores = {"Alice": 88, "Bob": 92,
     "Charlie": 85}
# dict mapping names to scores

student-record = {"name": "Alice",
     "age": 20,
     "scores": [88, 90, 85],
     "passed": True,
     "extra": None


# dict with mixed literals
```

GROUP 2

Python Variables and Attributes

Local variables exist inside functions and are limited to that block.

Instance attributes belong to each object and hold unique data.

Class attributes belong to the class and are shared by all instances.

```
class Student:
    school_name = "Tech Institute"
    # Class attribute shared by all

    def __init__(self, name): self.name =
    name
    # Instance attribute unique to each
    student

alice = Student("Alice") bob =
Student("Bob")

print(alice.name)          # Output: Alice
print(bob school_name)     # Output: Tech Institute
```

GROUP 3

Python Modifiers and Access Control

(PART 1)

Naming conventions define access

Public names are open; single underscore signals non-public; double underscore marks private.

Access control by convention

Python relies on naming conventions and name mangling to restrict direct access to private data.

@staticmethod decorator

Defines a method that does not receive instance or class reference; acts like a regular function.

@classmethod decorator

Defines a method receiving the class as the first argument; ideal for factory methods and class-wide behavior.

```
class BankAccount:
        def _init_(self, owner, balance=0):
            self._owner = owner          # Private attribute
            self._balance = balance      # Private attribute

    @staticmethod
    def bank_policy():
        return "Standard policy applies."

    @classmethod
    def create-account(cls, owner):
        return cls(owner, 0)

    def deposit(self, amount):
        if amount > 0:
            self._balance = amount
```

GROUP 4

Python Modifiers and OOP Features

PART 2

Abstract classes and methods

Define interfaces with abstract methods that subclasses must implement.

Access control and encapsulation

Bundle data and methods together to restrict direct access and protect object integrity.

Other OOP features

Use @property for managed attributes and @final to prevent subclassing or overriding.

```
from abc import ABC, abstractmethod
from typing import final

class Shape(ABC):
@abstractmethod
  def area(self):
    pass

class Circle(Shape):
    def _init_(self, radius):
      self.radius = radius
    def area(self):
      return 3.1416 * self.radius ** 2

@final
class Rectangle(Shape):
    def _init_(self, width, height):
      self.width = width
      self.height = height
@property
def area(self):
    return self.width * self.height
```

GROUP 5

Python Operators (Part 1)

Perform calculations, compare values, manipulate bits, and combine conditions.

Arithmetic Operators	+ (add), - (subtract), * (multiply), / (divide float), // (floor division), % (modulus), ** (power)
Comparison Operators	== (equal), != (not equal), >, <, >=, <=
Bitwise Operators	& (and), | (or), ^ (xor), ~ (not), << (left shift), >> (right shift)
Logical Operators	**and, or, not** (boolean logic combining conditions)

```
score = 85
total = 100

percentage = (score / total) * 100          # Arithmetic

passed = percentage >= 60                   # Comparison

honor_roll = passed and score > 90          # Logical

bit_mask = 0b1100 & 0b1010                  # Bitwise AND
```

GROUP 6

Python Operators (Part 2)

Assignment operators

Update values in place with concise notation.

Special operators

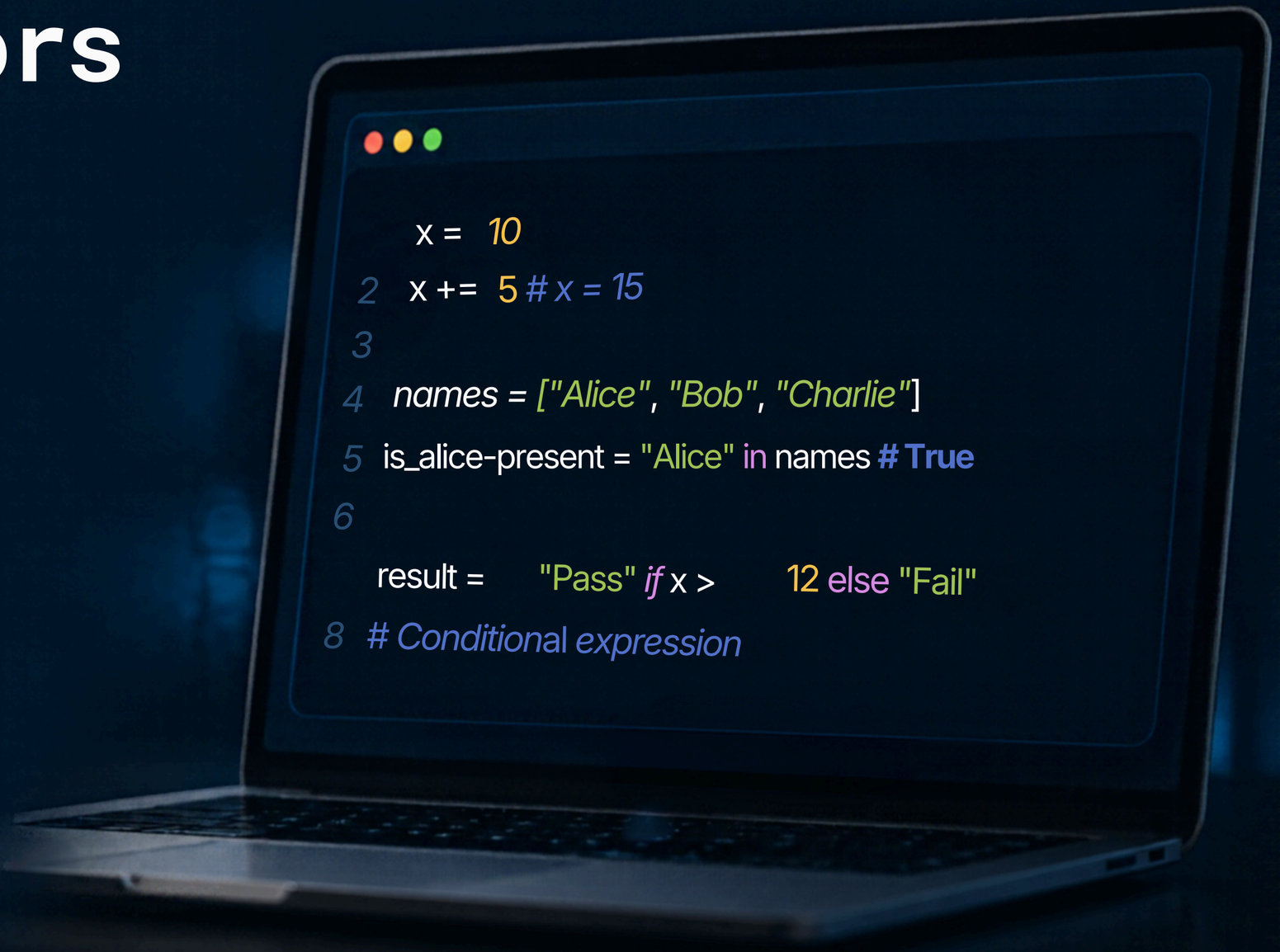
Identity: **is**, is not

Membership: **in**, not in

Conditional expressions: **x if condition** else y

Operator precedence & associativity

Define the order and grouping of evaluation.



GROUP 7

Python Decision-Making Statements

if statement

Runs a block of code only when the condition is True.

if-else statement

Chooses between two blocks based on the condition.

if-elif-else statement

Checks multiple conditions in order and runs the first true branch.

```
score = 75
if score >= 90:  90:
    grade = "A"'A'
elif score >=     80:
    grade = "B"'B'
elif score >=     70:
    grade = "C"'C'
elif score >=     60:
    grade = "D"'D'
else:
    grade = "F"'F'

print(f"Student grade: {grade}")
```

GROUP 8 & 9

Python Decision-Making (Part 2) and Loops and Loop Control

Make complex choices

nested if statements and structured branching with if-elif-else or match-case

Write compact logic

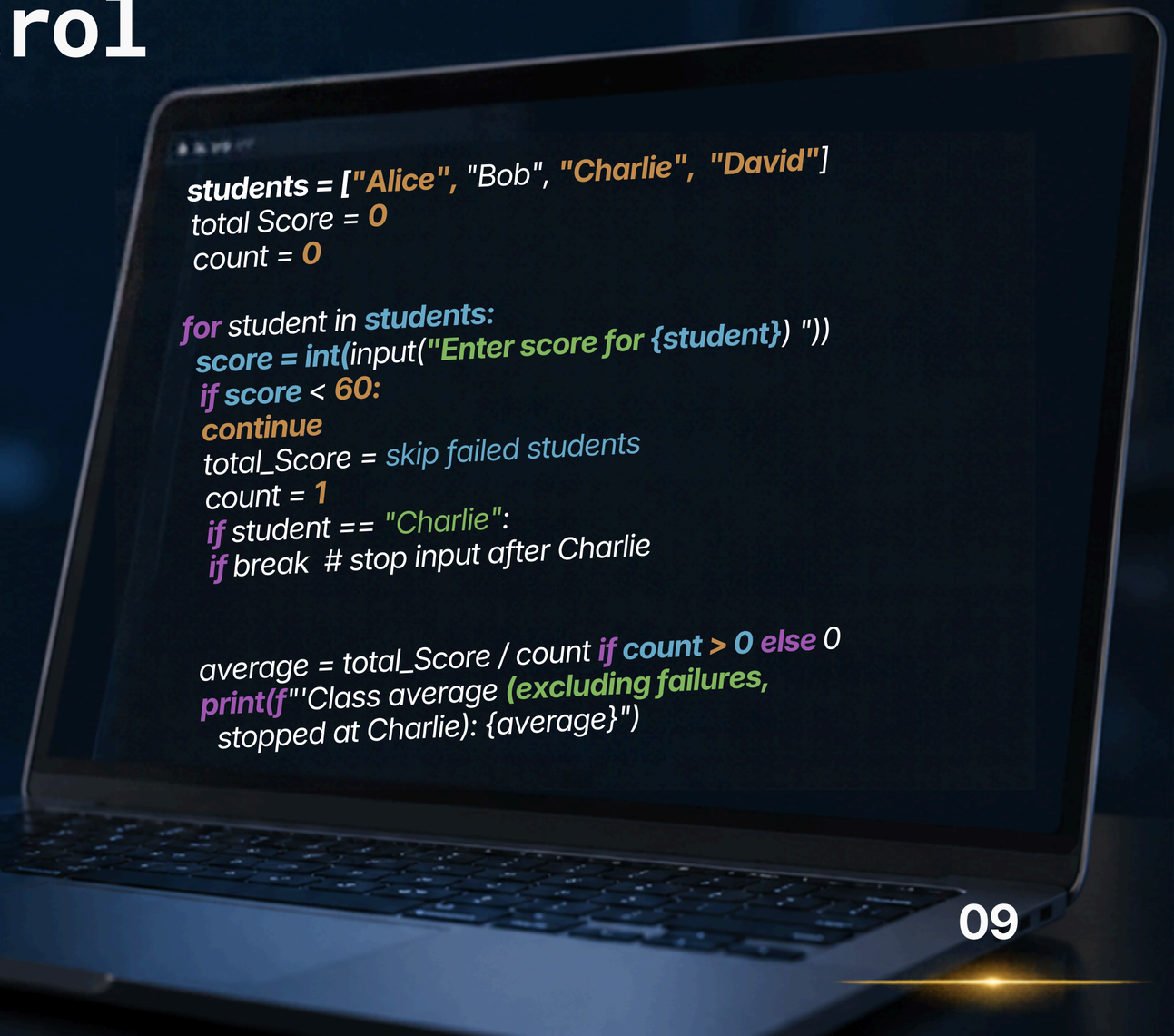
use conditional expressions for inline decisions

Repeat with confidence

while loops for conditions, for loops for iterables mimic do-while behavior when needed

Control your loops

use break to exit, continue to skip, pass as a placeholder
break exits the nearest loop; Python has no native do-while



Thank You

Institute of Technology of Cambodia

CHHEAN RAVY

e20241154

I3_AMS_1A